Diagnóstico y Plan de Completado — IRPF Parser Platform

Fecha: 2026-03-05
Repositorio: `moimene/irpf-parser-platform`
Objetivo: Completar el flujo de ingesta de PDFs bancarios → extracción estructurada → validación fiscal → exportación AEAT

---

1. Estado Real del Sistema (Auditoría Completa)

Lo que SÍ está construido y funciona

La plataforma tiene una base sólida. La infraestructura de orquestación está completa: los contratos API entre Vercel, n8n y Railway están definidos y desplegados, Supabase tiene el esquema `irpf_*` aplicado con las 7 tablas necesarias, y el ciclo de eventos `parse.started → parse.completed → parse.failed → manual.review.required` funciona de extremo a extremo. El frontend tiene las pantallas de dashboard, expediente, cola de revisión y exportación. Los tests e2e pasan en verde.

Componente	Estado	Calidad
Contratos API (intake, parse, webhooks, exports)	✅ Completo	Producción
Schema Supabase `irpf_*` (7 tablas)	✅ Completo	Producción
Workflow n8n (8 nodos, eventos parse.*)	✅ Completo	Producción
UI: dashboard, expediente, review board	✅ Completo	MVP
Motor de reglas (FIFO, recompras 2/12m)	✅ Completo	MVP
Tests e2e (2/2 en verde)	✅ Completo	MVP

El Gap Crítico: el Parser Engine es un Esqueleto

El archivo `services/parser/app/parser_engine.py` es el corazón del sistema y está **incompleto de forma estructural**. Tiene la arquitectura correcta (3 niveles: plantilla → semántico → manual) pero la implementación real de extracción es mínima:

Problema 1 — Extracción de PDF real no implementada. El método `decode_pdf_base64()` usa `pypdf` para extraer texto plano, lo que funciona solo con PDFs nativos (texto seleccionable). Los extractos bancarios de Goldman Sachs, Pictet y Citi son frecuentemente PDFs escaneados o con texto en capas de imagen. Para esos documentos, `pypdf` devuelve cadena vacía y el parser cae directamente a revisión manual.

Problema 2 — Extracción por keywords es frágil. El motor de plantillas busca palabras clave (`dividend`, `dividendo`, `interest earned`...) línea a línea con regex. Esto falla ante:

- Tablas multi-columna donde la fecha está en una columna y el importe en otra
- Texto con saltos de línea en medio de un registro
- Importes en formato anglosajón (`1,234.56`) vs. europeo (`1.234,56`)
- Campos ISIN, ticker, divisa y retención que no se extraen en absoluto

Problema 3 — No hay persistencia de resultados del parser. Cuando el parser devuelve registros extraídos, la API de intake llama a `processWithParser()` pero **no guarda los `records` en `irpf_extractions` ni en `irpf_operations`**. Los datos se pierden. La tabla `irpf_operations` existe en Supabase pero nunca se escribe desde el flujo automático.

Problema 4 — El intake no acepta ficheros reales. El `IntakeForm` del frontend envía solo el nombre del fichero (`filename`), no el contenido binario. No hay un campo de subida de archivo real (`<input type="file">`), ni integración con Supabase Storage para guardar el PDF antes de parsearlo.

Problema 5 — El workflow n8n no llama al parser. El nodo `Call Railway Parser` en el workflow existe pero está configurado con una URL placeholder. El flujo real es: n8n recibe evento → llama al parser de Railway → devuelve resultado a Vercel. Pero el parser necesita el contenido del PDF, que actualmente no se pasa a través de n8n.

---

2. Mapa de Gaps por Prioridad

```
CRÍTICO (bloquea el flujo completo)
├── [G1] Subida real de PDF a Supabase Storage
├── [G2] OCR real: pdfplumber + fallback LLM para PDFs escaneados
├── [G3] Extracción estructurada por entidad (Pictet, GS, Citi)
└── [G4] Persistencia de records en irpf_extractions + irpf_operations

IMPORTANTE (flujo funciona pero incompleto)
├── [G5] n8n: pasar content_base64 del PDF al parser de Railway
├── [G6] UI: input de archivo real con drag-and-drop
└── [G7] Review board: aprobar/rechazar registros individuales

MEJORA (calidad y robustez)
├── [G8] Extracción de ISIN, divisa, retención, tipo de cambio
├── [G9] Detección de entidad por cabecera de PDF (no solo filename)
└── [G10] Evaluación automática contra goldens en CI
```

---

3. Plan de Completado — 4 Sprints

Sprint 1 — Flujo de ingesta real (3-4 días)

Objetivo: Un PDF real de Goldman Sachs puede subirse desde el navegador, guardarse en Supabase Storage, y llegar al parser con su contenido.

Cambios en `apps/web`:

Reemplazar el `IntakeForm` de texto plano por un componente con `<input type="file" multiple accept=".pdf">`. Al seleccionar archivos, el frontend los sube a Supabase Storage (`irpf-documents/{expediente_id}/{uuid}.pdf`) y envía la URL firmada al endpoint de intake. El endpoint de intake descarga el PDF desde Storage, lo codifica en base64 y lo pasa al parser.

```
// apps/web/app/api/documents/intake/route.ts — cambio clave
// Antes: solo recibe filename
// Después: recibe storage_path, descarga el PDF, lo convierte a base64
const { data } = await supabase.storage
  .from('irpf-documents')
  .download(document.storage_path);
const buffer = await data.arrayBuffer();
const contentBase64 = Buffer.from(buffer).toString('base64');
```

Cambios en `services/parser`:

Añadir `pdfplumber` como dependencia principal de extracción (mucho más preciso que `pypdf` para tablas financieras). Mantener `pypdf` como fallback.

```
# pyproject.toml
dependencies = [
  "fastapi>=0.115.0",
  "uvicorn[standard]>=0.32.0",
  "pydantic>=2.10.0",
  "pypdf>=5.0.0",
  "pdfplumber>=0.11.0",   # ← NUEVO: extracción tabular precisa
  "pillow>=10.0.0",        # ← NUEVO: para PDFs con imágenes
]
```

Sprint 2 — Parser Engine real por entidad (4-5 días)

Objetivo: Para Pictet, Goldman Sachs y Citi, el parser extrae correctamente fecha, importe, ISIN, divisa, retención y tipo de operación con confianza ≥ 0.90.

El `parser_engine.py` necesita reescribirse con extractores específicos por entidad. La clave es que cada banco tiene un formato de tabla diferente:

Pictet — Extractos en formato tabla con columnas fijas: `Fecha | Descripción | ISIN | Cantidad | Precio | Importe | Divisa`. `pdfplumber` puede extraer estas tablas directamente con `page.extract_tables()`.

Goldman Sachs — Extractos en inglés con secciones separadas por tipo (Dividends, Interest, Realized Gains). Cada sección tiene un formato de tabla diferente. La

estrategia es detectar la sección por su cabecera y luego parsear las filas.

Citi — Formato más variable, mezcla de texto narrativo y tablas. Requiere combinar extracción de tablas con regex sobre el texto.

```
# services/parser/app/extractors/pictet.py — nuevo archivo
import pdfplumber
from typing import List
from app.schemas import ParsedRecord

def extract_pictet(pdf_bytes: bytes) -> List[ParsedRecord]:
    records = []
    with pdfplumber.open(BytesIO(pdf_bytes)) as pdf:
        for page in pdf.pages:
            tables = page.extract_tables()
            for table in tables:
                for row in table:
                    record = parse_pictet_row(row)
                    if record:
                        records.append(record)
    return records

def parse_pictet_row(row: List[str]) -> Optional[ParsedRecord]:
    # Detectar filas de dividendo por columna de descripción
    if not row or len(row) < 5:
        return None
    desc = str(row[1] or '').lower()
    if not any(k in desc for k in ['dividend', 'dividendo', 'interest',
'interes']):
        return None
    # Extraer campos
    date = parse_date(str(row[0] or ''))
    isin = extract_isin(str(row[2] or ''))
    amount = parse_amount(str(row[4] or ''))
    currency = str(row[5] or 'EUR').strip()
    record_type = 'DIVIDENDO' if 'div' in desc else 'INTERES'
    ...
```

Fallback LLM para documentos desconocidos:

Para entidades no reconocidas o PDFs con baja extracción textual, añadir un nivel 3 que llama a la API de OpenAI con el texto extraído y un prompt estructurado:

```python
# services/parser/app/extractors/llm_fallback.py
import os, json
import httpx

LLM_PROMPT = """Eres un extractor de datos financieros para IRPF español.
Del siguiente texto de extracto bancario, extrae TODAS las operaciones en
formato JSON:
[{"tipo": "DIVIDENDO|INTERES|VENTA|COMPRA", "fecha": "YYYY-MM-DD",
  "isin": "...", "importe": 0.00, "divisa": "EUR", "retencion": 0.00}]
Texto:
{text}
Responde SOLO con el array JSON, sin explicación."""

async def extract_with_llm(text: str) -> List[dict]:
    api_key = os.environ.get("OPENAI_API_KEY", "")
    if not api_key or len(text) < 50:
        return []
    async with httpx.AsyncClient() as client:
        resp = await client.post(
            "https://api.openai.com/v1/chat/completions",
            headers={"Authorization": f"Bearer {api_key}"},
            json={
                "model": "gpt-4o-mini",
                "messages": [{"role": "user", "content":
LLM_PROMPT.format(text=text[:4000])}],
                "temperature": 0,
                "response_format": {"type": "json_object"}
            },
            timeout=30
        )
        ...
```

Sprint 3 — Persistencia completa y Review Board funcional (3-4 días)

Objetivo: Los registros extraídos se guardan en Supabase y el fiscalista puede aprobar/rechazar cada uno desde la UI.

Cambios en el webhook handler (`/api/webhooks/parse-event`):

Cuando llega el evento `parse.completed`, el payload debe incluir los `records` extraídos. El webhook los guarda en `irpf_extractions` (el payload completo) y en

`irpf_operations` (una fila por registro validado):

```
// Al recibir parse.completed con records
if (parsed.data.event_type === 'parse.completed' &&
parsed.data.payload?.records) {
  const records = parsed.data.payload.records as ParsedRecord[];

  // 1. Guardar extracción completa
  await supabase.from(dbTables.extractions).insert({
    id: crypto.randomUUID(),
    document_id: parsed.data.document_id,
    raw_payload: { records },
    confidence: parsed.data.payload.confidence,
    requires_manual_review: parsed.data.payload.requires_manual_review,
  });

  // 2. Crear operaciones para registros de alta confianza
  const highConfidenceRecords = records.filter(r => r.confidence >= 0.85);
  for (const record of highConfidenceRecords) {
    await supabase.from(dbTables.operations).insert({
      id: crypto.randomUUID(),
      expediente_id: resolvedExpediente.id,
      document_id: parsed.data.document_id,
      operation_type: record.record_type,
      operation_date: record.fields.operation_date,
      isin: record.fields.isin,
      realized_gain: record.fields.amount,
      confidence: record.confidence,
      source: 'AUTO',
    });
  }
}
```

Cambios en el Review Board:

El `ReviewBoard` actual solo muestra el estado del documento. Necesita mostrar los registros individuales de cada extracción y permitir aprobar/rechazar:

```
// apps/web/components/review-board.tsx — nueva sección
function ExtractionReview({ documentId }: { documentId: string }) {
  const [records, setRecords] = useState<ParsedRecord[]>([]);

  async function approve(recordId: string) {
    await fetch(`/api/extractions/${recordId}/approve`, { method: 'POST' });
  }

  async function reject(recordId: string, reason: string) {
    await fetch(`/api/extractions/${recordId}/reject`, {
      method: 'POST',
      body: JSON.stringify({ reason })
    });
  }

  return (
    <table>
      {records.map(r => (
        <tr key={r.id}>
          <td>{r.record_type}</td>
          <td>{r.fields.operation_date}</td>
          <td>{r.fields.isin}</td>
          <td>{r.fields.amount}</td>
          <td>{(r.confidence * 100).toFixed(0)}%</td>
          <td>
            <button onClick={() => approve(r.id)}>✓ Validar</button>
            <button onClick={() => reject(r.id, '')}>✗ Rechazar</button>
          </td>
        </tr>
      ))}
    </table>
  );
}
```

Sprint 4 — Exportación AEAT real y cierre (3-4 días)

Objetivo: El expediente genera un fichero `.100` o `.720` válido para importar en el portal de la AEAT.

El endpoint de exportación actual devuelve un JSON lógico. Para generar el fichero real en formato AEAT (texto plano con registros de longitud fija), añadir un generador específico por modelo:

```
// apps/web/lib/exporters/modelo-720.ts
export function generateModelo720(operations: Operation[]): string {
  const lines: string[] = [];
  // Registro tipo 1: Identificación del declarante
  lines.push(buildTipo1Record(operations));
  // Registros tipo 2: Una línea por bien/derecho en el extranjero
  for (const op of operations.filter(o => o.operation_type === 'POSICION'))
{
    lines.push(buildTipo2Record(op));
  }
  return lines.join('\r\n');
}

function buildTipo2Record(op: Operation): string {
  // Formato AEAT: campos de longitud fija según BOE-A-2013-2154
  return [
    '720',           // Modelo
    '2025',          // Ejercicio
    padLeft(op.nif, 9),
    padRight(op.isin, 12),
    padRight(op.description, 40),
    padLeft(String(Math.round(op.value_eur * 100)), 18),
    // ... resto de campos según especificación AEAT
  ].join('');
}
```

---

4. Variables de Entorno Adicionales Necesarias

Variable	Servicio	Uso
`OPENAI_API_KEY`	Railway (parser)	Fallback LLM para PDFs desconocidos
`SUPABASE_STORAGE_BUCKET`	Vercel	Bucket para PDFs subidos
`PARSER_MAX_PDF_SIZE_MB`	Railway	Límite de tamaño (recomendado: 50MB)

---

5. Orden de Implementación Recomendado

El orden importa porque cada sprint desbloquea el siguiente:

1. **Sprint 1 primero** — Sin subida real de PDF, no hay nada que parsear. Es el desbloqueador de todo.
2. **Sprint 2 segundo** — El parser real es el corazón del valor. Sin él, el sistema solo mueve metadatos.
3. **Sprint 3 tercero** — La persistencia de resultados convierte el parser en una herramienta de trabajo real.
4. **Sprint 4 último** — La exportación AEAT es el entregable final para el fiscalista.

Estimación total: 13-17 días de desarrollo con un ingeniero dedicado, o 7-9 días con dos ingenieros en paralelo (Sprint 1+2 simultáneos, Sprint 3+4 en secuencia).

---

6. Decisión Clave: ¿LLM o Plantillas para el Parser?

Esta es la decisión de arquitectura más importante del proyecto. Hay dos estrategias:

Estrategia A — Plantillas deterministas (pdfplumber + regex): Alta precisión para entidades conocidas (Pictet, GS, Citi), coste cero de inferencia, reproducible y auditable. Falla ante formatos nuevos o PDFs de baja calidad. **Recomendada para el 80% de los documentos.**

Estrategia B — LLM semántico (GPT-4o-mini): Funciona con cualquier formato, extrae campos implícitos, maneja texto en inglés/alemán/francés sin configuración. Coste ~0.002€/documento, latencia 3-8 segundos, no determinista. **Recomendada como fallback para el 20% restante.**

La arquitectura de 3 niveles ya definida en el código es la correcta: Nivel 1 (plantilla) → Nivel 2 (LLM semántico) → Nivel 3 (revisión manual). Solo falta implementar el Nivel 1 con `pdfplumber` y el Nivel 2 con GPT-4o-mini.